SkillProof Platform

Project-Based Career Evidence & Skill Verification System

Technical Specification, Engine Formulas, and Execution Guide

Release: v1.0.0 Production	**Tech Stack:** FastAPI / SQLite / ES6
Author: SkillProof Architecture Team	**Verification:** 13/13 Automated Tests Pass
App URL: http://127.0.0.1:8000/	**API Docs:** http://127.0.0.1:8000/docs

1. Executive Summary & Problem Statement

Modern technical hiring is broken by credential inflation. Traditional resumes, keyword lists, and course certificates of completion fail to reflect authentic software engineering capability. Recruiters are inundated with applicants claiming identical skill sets, while talented students struggle to substantiate their competence.

SkillProof solves this through **Project-Based Career Evidence**. By ingesting public GitHub repositories and analyzing actual commit cadence, codebase complexity, language distribution, and documentation quality, SkillProof produces a transparent **0–100 Evidence Score**. Automated insights are coupled with faculty/mentor peer review to deliver tamper-resistant skill verification.

2. System Architecture & Component Design

- **Presentation Layer:** Single-Page Application using HTML5, modern CSS3 dark-mode glassmorphism, and responsive vanilla ES6 JavaScript. Visualizes commit activity using Chart.js line charts and vector icons via Lucide.
- **Application / API Server:** FastAPI asynchronous microframework handling RESTful routing, input validation with Pydantic v2 schemas, and interactive OpenAPI documentation.
- **Data Persistence Layer:** SQLite database backed by SQLAlchemy ORM with canonical absolute path resolution for reliable multi-process operations.
- **Deterministic Scoring Engine:** Multi-variable mathematical pipeline evaluating commit consistency, repo complexity, README quality, recent activity, and anti-gaming anomalies.
- **AI Summary & Skill Extraction:** Hybrid intelligent parser extracting skills from language profiles, topics, and README keywords with OpenAI API support and rule-based fallback.

3. Deterministic Evidence Scoring Engine

The Evidence Score evaluates repos across 5 key pillars (max 100 pts) followed by anti-gaming penalty subtractions:

Pillar	Max Pts	Mathematical Logic & Criteria
Commit Consistency	30 pts	Evaluates sustained development across months. Diminishing returns on commit count (up to 15 pts) + consistency ratio across active months (up to 15 pts).

Project Complexity	25 pts	Language distribution balance (up to 10 pts for 3+ languages), file count depth (up to 8 pts for 30+ files), and code size (up to 7 pts for >500KB).
README Quality	20 pts	Length score (up to 10 pts for >1500 chars) + quality markers (install, usage, code blocks, headers, license, contributing).
Recent Activity	15 pts	Recency of last commit: 15 pts if <30 days; 10 pts if <90 days; 6 pts if <180 days; 2 pts if >1 year.
Community Signal	10 pts	Repository stars (up to 6 pts) and originality verification (original repo vs original commits in fork).

Anti-Gaming Safeguards:

- **Single-Day Burst Penalty (-15 pts):** Penalizes repos where all commits were dumped in a single 24h period.
- **Bare Fork Penalty (-30 pts, capped at 10):** Penalizes forks that contain zero original commit contributions.
- **Student Score Aggregation:** Computes a weighted average favoring the top 3 best projects, plus multi-project quality bonuses (up to +10 pts) for students with multiple verified projects.

4. Database Models & Schema Design

- **Student:** Stores user profile, email, GitHub handle, evidence score, and JSON score breakdown.
- **Project:** Stores repo metadata, languages byte dict, commit timeline array, README content, AI summary, and anti-gaming audit flags.
- **Skill:** Stores skill name, proficiency level, evidence source (AI-suggested vs verified), and verifying mentor name.
- **MentorFeedback:** Stores project feedback, 1-5 star ratings, qualitative strengths/improvements, and verified skills.

5. API Route Specifications

Endpoint	Method	Function & Purpose
/api/health	GET	Service health check status confirmation
/api/students/	GET	List all students with brief summary & top skills
/api/students/{id}	GET	Full student portfolio, commit timelines, and audit
/api/students/{s_id}/projects/{p_id}/visibility	PUT	Toggle project visibility (Public vs Private privacy control)
/api/recruiter/search	GET	Filter candidates by skill name and min score slider
/api/recruiter/skills	GET	List all unique skills with total and verified counts
/api/feedback/	POST	Submit mentor review & verify candidate skills

6. User Personas & Workflows

Student: Imports GitHub repositories, visualizes score breakdown, toggles project privacy, and shares verifiable link.
Recruiter: Searches candidate talent pool by verified skill badges and minimum evidence score filter.
Mentor: Conducts peer review, rates projects 1-5 stars, writes qualitative evaluations, and certifies skills.

7. Quick Execution Guide

Refer to **HOW_TO_RUN.txt** for comprehensive instructions. Key execution steps:

```
# 1. Install dependencies
pip install -r backend/requirements.txt

# 2. Seed database with demo candidates
python -m app.seed

# 3. Run automated tests (13/13 pass)
pytest backend/tests -v

# 4. Launch web application server
uvicorn app.main:app --host 127.0.0.1 --port 8000

# 5. Access in browser: http://127.0.0.1:8000/
```